Homework 8 - Rental Housing Listings

Pragya Apurva | SID4 6102 | November 2, 2026

Repository: https://github.com/ishuapurva1996/data260-6102

Section 0 - Personal configuration

Value	Assigned result	Value	Assigned result
SID4	6102	PORT_BASE	8702
PREFIX	s6102	SEED	6102
VERIFY_SEED	266102	DOMAIN_ID	6 - Rental Housing

Hardware: Apple M4 MacBook Air, 24 GiB RAM. Python 3.12.13; Kafka 3.9.2. **Local model:** qwen2.5:7b Q4_K_M, temperature 0, seed 6102, context 4096, output cap 256, model timeout 180 seconds.

Tested code, tag hw8:
`8caf32f200a8411db513a784e2f3f7073e8339c5`

The repository link and collaborator access for Sbnikitha and supriyaselvanganesan were verified October 29, 2026. The dated receipt is in raw/collaborator_access.json.

Part 1 - Rental request and response

The client sends to `s6102_entity_requests`; the worker replies through `s6102_entity_responses`. The caller's correlation_id stays unchanged.

```
Request: {"correlation_id":"<caller ID>",
 "reply_to":"s6102_entity_responses",
 "operation":"CREATE_RENTAL","data":{...rental fields...}}
CREATE reply: {"correlation_id":"<same ID>","success":true,
 "operation":"CREATE_RENTAL","rental_id":"<UUID>",
 "message":"Rental created"}
GET uses operation="GET_RENTAL", data={"rental_id":"<UUID>"}.
GET reply: correlation_id, operation, success=true, rental={...}.
Error reply: correlation_id, operation, success=false, error.
```

Required fields: listingTitle, propertyAddress, submitterEmail, description, propertyType, and termsAccepted. Existing validation rejects empty or invalid data; availableUnits defaults to 1. The worker uses a plain Python dictionary and generates UUID v4 IDs.

Part 1 - Docker services and Kafka topics

```
Pragya Apurva | DATA 260 Homework 8

NAMES                  STATUS                       PORTS
s6102-hw8-fastapi-1    Up 12 minutes (healthy)      127.0.0.1:8702->8702/tcp
s6102-hw8-worker-1     Up 9 minutes
s6102-hw8-kafka-1      Up 13 minutes (healthy)      127.0.0.1:9092->9092/tcp
(base) pragyaapurva@Pragyas-MacBook-Air ~ %
```

Figure 1. Kafka, worker, and FastAPI are running on the assigned ports.

```
Pragya Apurva | DATA 260 Homework 8 | Startup logs

{"assignments": [{"partition": 0, "topic": "s6102_entity_requests"}], "event": "worker_ready", "group_id": "s6102_entity_worker", "idempotency": true, "message":
"Consumer is ready and listening...", "pid": 1, "source_commit": "8caf32f200a8411db513a784e2f3f7073e8339c5", "stage": "entity_worker", "student": "Pragya Apurva",
 "timestamp": "2026-11-02T08:06:23.509686+00:00", "topic": "s6102_entity_requests"}
{"bind": "0.0.0.0:8702", "event": "web_host_url", "pid": 1, "source_commit": "8caf32f200a8411db513a784e2f3f7073e8339c5", "student": "Pragya Apurva", "timestamp":
"2026-11-02T08:02:49.099731+00:00", "url": "http://127.0.0.1:8702"}
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8702 (Press CTRL+C to quit)
(base) pragyaapurva@Pragyas-MacBook-Air ~ %
```

Figure 2. Worker readiness and FastAPI startup at http://127.0.0.1:8702.

Startup confirms **Consumer is ready and listening...** and **http://127.0.0.1:8702**.

```
Pragya Apurva | DATA 260 Homework 8 | Kafka topics

s6102_drafts               partitions=1 replicas=1
s6102_dead_letter          partitions=1 replicas=1
s6102_entity_requests      partitions=1 replicas=1
s6102_final                partitions=1 replicas=1
s6102_entity_responses     partitions=1 replicas=1
s6102_inbox                partitions=1 replicas=1
s6102_tasks                partitions=2 replicas=1
(base) pragyaapurva@Pragyas-MacBook-Air ~ %
```

Figure 3. Seven topics; s6102_tasks has two partitions. All replication factors are 1.

Part 1 - VS Code implementation snippets

```python
        response = {
            "correlation_id": envelope.correlation_id,
            "operation": envelope.operation,
            "success": True,
        }
        if envelope.operation == "CREATE_RENTAL":
            rental_id = str(uuid4())
            self.rentals[rental_id] = {"rental_id": rental_id, **data}
            response.update(rental_id=rental_id, message="Rental created")
        else:
            rental = self.rentals.get(data["rental_id"])
            if rental is None:
                return error_response(payload, "Rental not found")
            response["rental"] = copy.deepcopy(rental)

        if self.idempotency:
            self.responses[identity] = (fingerprint, copy.deepcopy(response))
        event(
            "request_processed",
```

Code 01. src/kafka_service/worker.py, lines 55-71: CREATE, GET, and response caching.

```python
            return error_response(payload, "Kafka RPC is not ready", transport=True)
        key = (envelope["correlation_id"], envelope["operation"])
        if key in self.pending:
            return error_response(payload, "This correlation ID and operation are already pending")
        future = asyncio.get_running_loop().create_future()
        self.pending[key] = future  # Register before a fast worker can reply.
        try:
            async with asyncio.timeout(self.settings.rpc_timeout):
                event(
                    "request_sending",
                    stage="rpc",
                    topic=ENTITY_REQUESTS,
                    message=envelope,
                    correlation_id=key[0],
                    operation=key[1],
                )
                sent = await self.producer.send_and_wait(
                    ENTITY_REQUESTS, json.dumps(envelope).encode()
                )
                event(
                    "request_sent",
                    stage="rpc",
                    topic=sent.topic,
                    partition=sent.partition,
                    offset=sent.offset,
                    correlation_id=key[0],
                    operation=key[1],
                )
                response = await future
                event(
                    "response_returned",
                    stage="rpc",
                    correlation_id=key[0],
                    operation=key[1],
                    response=response,
                )
                return response
        except TimeoutError:
```

Code 02. src/kafka_service/kafka_rpc.py, lines 137-172: register, send, await, and return.

Part 1 - CREATE and GET results

homework_demo.py creates a rental, retrieves it using the returned UUID, and checks the same correlation ID and rental fields.

```
Pragya Apurva | DATA 260 Homework 8
LIVE CREATE_RENTAL -> GET_RENTAL | HTTP + Kafka

Observed: 2026-11-02T08:24:48.350259+00:00
Base URL: http://127.0.0.1:8702 | authenticated writer
Caller correlation_id: 2ca087a3-99ad-4868-ad02-ab077d03b5f5

CREATE_RENTAL request data:
  listingTitle: HW8 synthetic apartment
  propertyAddress: 100 Example Street, San Jose, CA
  submitterEmail: hw8@example.com
  description: Synthetic rental listing for the Kafka homework demonstration.
  propertyType: apartment
  termsAccepted: true
CREATE_RENTAL response:
  success: true
  rental_id: ebc9fc34-6fc3-44ac-b4b2-601995ac6c41
  correlation_id: 2ca087a3-99ad-4868-ad02-ab077d03b5f5
  message: Rental created

GET_RENTAL request:
  rental_id: ebc9fc34-6fc3-44ac-b4b2-601995ac6c41
GET_RENTAL response:
  success: true
  correlation_id: 2ca087a3-99ad-4868-ad02-ab077d03b5f5
  rental_id: ebc9fc34-6fc3-44ac-b4b2-601995ac6c41
  listingTitle: HW8 synthetic apartment
  propertyAddress: 100 Example Street, San Jose, CA
  availableUnits: 1
  submitterEmail: hw8@example.com
  description: Synthetic rental listing for the Kafka homework demonstration.
  propertyType: apartment
  termsAccepted: true

Assertions passed: UUID v4, same correlation_id, matching rental fields.
Credentials and bearer tokens are omitted.
(base) pragyaapurva@Pragyas-MacBook-Air report-capture-20261102 %
```

Figure 4. CREATE and GET succeed with the same UUID and correlation ID.

Validation results

Input	Observed result
Missing required fields	success=false; named field errors
termsAccepted=false	success=false; acceptance error
Unknown operation	success=false; Unknown operation
Unknown rental UUID	success=false; Rental not found

Each error preserves its caller correlation_id. Full request/reply records: raw/part1/validation.json.

Part 1 - Complete CREATE to GET flow

The same journey ID links the request, worker processing, reply, and final client response for both operations.

```
Pragya Apurva | DATA 260 Homework 8
COMPLETE CREATE -> GET MESSAGE FLOW | actual process logs

correlation_id: 2ca087a3-99ad-4868-ad02-ab077d03b5f5
rental_id: ebc9fc34-6fc3-44ac-b4b2-601995ac6c41
UTC times; grouped by operation and message hop.

CREATE_RENTAL
08:24:48.334 client demo_request -> s6102_entity_requests
08:24:48.337 rpc request_sending -> s6102_entity_requests
08:24:48.338 rpc request_sent -> s6102_entity_requests p0/o71
08:24:48.338 entity_worker request_received -> s6102_entity_requests p0/o71
08:24:48.338 entity_worker request_processed ->
08:24:48.339 entity_worker response_sent -> s6102_entity_responses p0/o75
08:24:48.339 rpc response_received -> s6102_entity_responses p0/o75
08:24:48.339 rpc response_returned -> caller
08:24:48.342 client demo_response -> caller

GET_RENTAL
08:24:48.342 client demo_request -> s6102_entity_requests
08:24:48.344 rpc request_sending -> s6102_entity_requests
08:24:48.345 rpc request_sent -> s6102_entity_requests p0/o72
08:24:48.346 entity_worker request_received -> s6102_entity_requests p0/o72
08:24:48.346 entity_worker request_processed ->
08:24:48.346 entity_worker response_sent -> s6102_entity_responses p0/o76
08:24:48.346 rpc response_received -> s6102_entity_responses p0/o76
08:24:48.346 rpc response_returned -> caller
08:24:48.349 client demo_response -> caller

Every hop above uses the same caller correlation_id.
Original timestamped records saved in raw/part1/user_flow.jsonl.
(base) pragyaapurva@Pragyas-MacBook-Air report-capture-20261102 %
```

Figure 5. Actual message hops, topic names, partitions, offsets, and UTC timestamps.

CREATE: request offset 71, reply offset 75. GET: request offset 72, reply offset 76. Original records: raw/part1/user_flow.jsonl.

Part 1 - Kafka client and matching replies

The client registers its pending result before sending and matches replies by correlation_id and operation.

```
Pragya Apurva | DATA 260 Homework 8
KAFKA CLIENT | relevant source and observed result

Send the caller envelope and await its complete response:
src/kafka_service/kafka_rpc.py
137         key = (envelope["correlation_id"], envelope["operation"])
138         if key in self.pending:
139             return error_response(payload, "This correlation ID and operation are already pending")
140         future = asyncio.get_running_loop().create_future()
141         self.pending[key] = future  # Register before a fast worker can reply.
    ...
152                 sent = await self.producer.send_and_wait(
153                     ENTITY_REQUESTS, json.dumps(envelope).encode()
154                 )
    ...
164                 response = await future
    ...
172                 return response

Match replies by correlation_id AND operation:
src/kafka_service/kafka_rpc.py
 88                     response = json.loads(record.value.decode())
 89                     if not isinstance(response, dict) or type(response.get("success")) is not bool:
 90                         raise ValueError("Response must contain Boolean success")
 91                     key = (response.get("correlation_id"), response.get("operation"))
 92                     future = self.pending.get(key)
    ...
113                 if future is not None and not future.done():
114                     future.set_result(response)

Observed authenticated CREATE/GET run:
correlation_id: 2ca087a3-99ad-4868-ad02-ab077d03b5f5
CREATE rental_id: ebc9fc34-6fc3-44ac-b4b2-601995ac6c41
GET rental_id: ebc9fc34-6fc3-44ac-b4b2-601995ac6c41
CREATE success=true | GET success=true
UUID, correlation preservation and field equality assertions: PASS
(base) pragyaapurva@Pragyas-MacBook-Air report-capture-20261102 %
```

Figure 6. Kafka client source above the successful CREATE/GET results.

Part 1 - homework_demo.py

One correlation ID is reused; GET uses the UUID returned by CREATE. Assertions verify success, UUID v4, and field equality.

```
Pragya Apurva | DATA 260 Homework 8
HOMEWORK_DEMO.PY | relevant source and observed result

scripts/hw08/homework_demo.py
 30 async def demonstrate(send, *, correlation_id=None, run_id=None):
 31     correlation_id = correlation_id or str(uuid4())
 32     run_id = run_id or f"part1-{uuid4().hex[:12]}"
 33     records = []
 34     for operation, data in (("CREATE_RENTAL", RENTAL), ("GET_RENTAL", None)):
 35         if operation == "GET_RENTAL":
 36             data = {"rental_id": records[0]["response"]["rental_id"]}
 37         request = {
 38             "correlation_id": correlation_id,
 39             "reply_to": ENTITY_RESPONSES,
 40             "operation": operation,
 41             "data": data,
 42         }
    ...
 51         response = await send(request)
    ...
 59         assert response["success"] is True, response
 60         assert response["correlation_id"] == correlation_id, response
 61         assert response["operation"] == operation, response
 62         records.append({"request": request, "response": response})
 63         if operation == "CREATE_RENTAL":
 64             assert UUID(response["rental_id"]).version == 4
 65         else:
 66             rental = response["rental"]
 67             assert rental["rental_id"] == records[0]["response"]["rental_id"]
 68             assert all(rental[field] == value for field, value in RENTAL.items())

Observed authenticated CREATE/GET run:
correlation_id: 2ca087a3-99ad-4868-ad02-ab077d03b5f5
CREATE rental_id: ebc9fc34-6fc3-44ac-b4b2-601995ac6c41
GET rental_id: ebc9fc34-6fc3-44ac-b4b2-601995ac6c41
CREATE success=true | GET success=true
UUID, correlation preservation and field equality assertions: PASS
(base) pragyaapurva@Pragyas-MacBook-Air report-capture-20261102 %
```

Figure 7. Demo code followed by the observed rental ID and passing assertions.

Part 1 - Worker storage and validation

The worker validates rental fields, stores rentals in its dictionary, and returns clean errors for an unknown ID.

```
Pragya Apurva | DATA 260 Homework 8
RENTAL WORKER | dictionary, validation and UUID CREATE/GET

src/kafka_service/worker.py
 24 class RentalWorker:
 25     def __init__(self, *, idempotency=True):
 26         self.rentals = {}  # Assignment requirement: worker-local plain Python dictionary.
 27         self.responses = {}
 28         self.idempotency = idempotency
    ...
 30     def process_request(self, payload) -> dict:
 31         try:
 32             envelope = RentalRequest.model_validate(payload)
 33             data = validate_data(envelope.operation, envelope.data)
 34         except ValidationError as exc:
 35             return error_response(payload, clean_validation_error(exc))
 36         except ValueError as exc:
 37             return error_response(payload, str(exc))
    ...
 60         if envelope.operation == "CREATE_RENTAL":
 61             rental_id = str(uuid4())
 62             self.rentals[rental_id] = {"rental_id": rental_id, **data}
 63             response.update(rental_id=rental_id, message="Rental created")
 64         else:
 65             rental = self.rentals.get(data["rental_id"])
 66             if rental is None:
 67                 return error_response(payload, "Rental not found")
 68             response["rental"] = copy.deepcopy(rental)

Observed authenticated CREATE/GET run:
correlation_id: 2ca087a3-99ad-4868-ad02-ab077d03b5f5
CREATE rental_id: ebc9fc34-6fc3-44ac-b4b2-601995ac6c41
GET rental_id: ebc9fc34-6fc3-44ac-b4b2-601995ac6c41
CREATE success=true | GET success=true
UUID, correlation preservation and field equality assertions: PASS
(base) pragyaapurva@Pragyas-MacBook-Air report-capture-20261102 %
```

Figure 8. Dictionary, validation, and CREATE/GET code followed by the matching results.

Rental data and the entity replay cache are in memory and are lost on entity-worker restart.

Part 1 - FastAPI wrapper and account roles

The existing JWT dependency protects POST /api/kafka/rentals. CREATE allows writer/moderator; GET accepts an authenticated reader.

```
Pragya Apurva | DATA 260 Homework 8
FASTAPI WRAPPER | existing JWT roles and Kafka lifecycle

code/web_application/routers/kafka_rentals.py
  9 def create_router(rpc):
 10     router = APIRouter(prefix="/api/kafka", tags=["HW8 Kafka rentals"])
 11
 12     @router.post("/rentals")
 13     async def rental_operation(payload: dict, user=Depends(require_user)):
 14         if payload.get("operation") == "CREATE_RENTAL" and user.role not in {"writer", "moderator"}:
 15             raise HTTPException(403, "Your role does not permit this action.")
 16         response = await rpc.make_request(payload)
 17         return JSONResponse(
 18             status_code=503 if response.get("transport_error") else 200, content=response
 19         )

code/web_application/main.py
 69         kafka_rpc = kafka_rpc if kafka_rpc is not None else KafkaRPC()
 70         app.state.kafka_rpc = kafka_rpc
 71         app.add_event_handler("startup", kafka_rpc.start)
 72         app.add_event_handler("shutdown", kafka_rpc.stop)
 73         app.include_router(kafka_rentals.create_router(kafka_rpc))

Actual HTTP acceptance results from 2026-11-02T08:06:23.851520+00:00
http_role_logins: PASS
  {"statuses": {"reader": 200, "writer": 200, "moderator": 200}}
http_missing_invalid_token: PASS
  {"statuses": [401, 401, 401]}
http_wrong_password: PASS
  {"status": 401}
authenticated_fastapi_wrapper: PASS
  {"correlation_id": "5e6d0a90-e63f-458f-a170-1818d14a742b", "rental_id":
  "9d01413e-5968-434c-9fae-418e89a2bb9d", "create_status": 200, "get_status": 200}
http_reader_permissions: PASS
  {"create_status": 403, "get_status": 200}
http_moderator_permissions: PASS
  {"create_status": 200, "get_status": 200}
(base) pragyaapurva@Pragyas-MacBook-Air report-capture-20261102 %
```

Figure 9. API wrapper and connection lifecycle code above the HTTP acceptance results.

Request or role	HTTP result
Reader / writer / moderator login	200 / 200 / 200
Missing or invalid token; wrong password	401
Reader CREATE / GET	403 / 200
Writer or moderator CREATE / GET	200 / 200

Part 2 - Kafka coordination

Planner, Writer, and Reviewer run as separate processes. Each reads its input topic and publishes to the next; no agent calls another directly.

```
Pragya Apurva | DATA 260 Homework 8
KAFKA AGENT WIRING | focused source and matching recorded processes

src/kafka_agents/common.py
 35 STAGES = {
 36     "planner": (INBOX, TASKS),
 37     "writer": (TASKS, DRAFTS),
 38     "reviewer": (DRAFTS, FINAL),
 39 }
    ...
367     store = ReceiptStore(Path(args.state_dir) / f"{stage}.sqlite3")
    ...
379     producer = AIOKafkaProducer(
380         bootstrap_servers=settings.bootstrap_servers, enable_idempotence=True
381     )
382     consumer = AIOKafkaConsumer(
383         bootstrap_servers=settings.bootstrap_servers,
384         group_id=processor.group_id,
385         client_id=processor.instance,
386         enable_auto_commit=False,
387         auto_offset_reset="earliest",
    ...
391     consumer.subscribe([STAGES[stage][0]], listener=AssignmentLogger(processor))
392     try:
393         await producer.start()
394         await consumer.start()
395         await wait_assignment(consumer, settings.assignment_timeout)
    ...
411         async for record in consumer:
412             await processor.handle(record, consumer, producer)
413     finally:
414         await consumer.stop()
415         await producer.stop()

Recorded real-model journey | model=qwen2.5:7b | fixture=false
planner  PID 88057 | s6102_inbox -> s6102_tasks
writer   PID 88058 | s6102_tasks -> s6102_drafts
reviewer PID 88059 | s6102_drafts -> s6102_final
Preserved correlation_id: 8b30711f-3912-40fa-9747-93b1ffa46d8a
Final status: approved | nonempty answer: True
(base) pragyaapurva@Pragyas-MacBook-Air report-capture-20261102 %
```

Figure 10. Kafka routes, consumer loop, and the three recorded process IDs.

s6102_inbox -> s6102_tasks -> s6102_drafts -> s6102_final. The same correlation ID is preserved throughout. SQLite files hold stage-local receipts; Kafka carries the handoffs.

Part 2 - Planner

Planner uses the real local qwen2.5:7b model to turn the rental-ordering question into a three-step plan.

```
Pragya Apurva | DATA 260 Homework 8
PLANNER | separate-process source and actual model output

src/kafka_agents/planner.py
  1 """Run the separate planner process."""
  2
  3 from .common import main
  4
  5 if __name__ == "__main__":
  6     main("planner")

src/kafka_agents/common.py
 36     "planner": (INBOX, TASKS),
    ...
146             result = await asyncio.wait_for(
147                 asyncio.to_thread(self.client.complete, messages),
148                 self.model_timeout + 5,
149             )
150             content = str(result.content).strip()
    ...
161         output = dict(data)
162         output["from_stage"] = self.stage
163         if self.stage == "planner":
164             output["plan"] = content
    ...
170         return output

Observed: 2026-11-02T08:06:35.305628+00:00
Observed PID: 88057 | model: qwen2.5:7b | fixture: false
Route: s6102_inbox -> s6102_tasks
correlation_id: 8b30711f-3912-40fa-9747-93b1ffa46d8a

Actual plan:
To understand how Kafka message ordering affects updates to the same rental listing,
follow these steps:

1. **Identify Message Order**: Determine the sequence in which messages are sent and
received in Kafka. Messages are ordered based on the partition they are published to.
2. **Track Listing Updates**: Monitor the updates to the rental listing as they are
published to Kafka. Ensure that updates are processed in the order they were sent to
maintain consistency.
3. **Verify Order Compliance**: Confirm that the system consuming messages from Kafka
processes the updates to the rental listing in the correct order, preventing any race
conditions or inconsistencies.
Published to: s6102_tasks partition=0 offset=47
Original full stage records: raw/part2/journey.jsonl
(base) pragyaapurva@Pragyas-MacBook-Air report-capture-20261102 %
```

Figure 11. Planner code and actual plan; published to s6102_tasks, partition 0, offset 47.

Part 2 - Writer

Writer consumes the plan and generates a draft using the real local model.

```
Pragya Apurva | DATA 260 Homework 8
WRITER | separate-process source and actual model output

src/kafka_agents/writer.py
  1 """Run the separate writer process."""
  2
  3 from .common import main
  4
  5 if __name__ == "__main__":
  6     main("writer")

src/kafka_agents/common.py
 37     "writer": (TASKS, DRAFTS),
    ...
161         output = dict(data)
162         output["from_stage"] = self.stage
    ...
165         elif self.stage == "writer":
166             output["draft"] = content
167             output["writer_instance"] = self.instance
    ...
170         return output

Observed: 2026-11-02T08:06:39.511149+00:00
Observed PID: 88058 | model: qwen2.5:7b | fixture: false
Route: s6102_tasks -> s6102_drafts
correlation_id: 8b30711f-3912-40fa-9747-93b1ffa46d8a

Actual draft:
Kafka ensures message ordering within partitions, which is crucial for maintaining
consistent updates to rental listings. By publishing updates to the same partition,
you ensure they are processed in the order they are sent. This prevents race
conditions and guarantees that updates to a listing are applied sequentially,
maintaining data integrity.
Published to: s6102_drafts partition=0 offset=98
Original full stage records: raw/part2/journey.jsonl
(base) pragyaapurva@Pragyas-MacBook-Air report-capture-20261102 %
```

Figure 12. Writer code and actual draft; published to s6102_drafts, partition 0, offset 98.

Part 2 - Reviewer and final JSON



Figure 13. Reviewer code and final record read from s6102_final, partition 0, offset 12.

Final result


```
{
  "correlation_id": "8b30711f-3912-40fa-9747-93b1ffa46d8a",
  "status": "approved",
  "answer": "Kafka ensures message ordering within partitions, which is crucial for maintaining consistent updates to rental listings. By publishing updates to the same partition, they are processed in the order they are sent, preventing race conditions and ensuring that updates to a listing are applied sequentially, maintaining data integrity."
}
```


The result preserves the journey ID. The application labels a successful Reviewer response approved.

Part 2 - Complete message journey

Three distinct processes record receive, model, publish, and commit events for one correlation ID.

```
Pragya Apurva | DATA 260 Homework 8
COMPLETE THREE-AGENT JOURNEY | recorded process logs

correlation_id: 8b30711f-3912-40fa-9747-93b1ffa46d8a
Observed run: 2026-11-02T08:06:24.743465+00:00
Real local model: qwen2.5:7b | three distinct recorded PIDs

planner  PID 88057 | s6102_inbox -> s6102_tasks
writer   PID 88058 | s6102_tasks -> s6102_drafts
reviewer PID 88059 | s6102_drafts -> s6102_final

Actual stage log events (UTC timestamp order):
08:06:24.743 planner  message_received     s6102_inbox p0/o4
08:06:24.745 planner  processing_started   s6102_inbox p0/o4
08:06:35.305 planner  model_result         model=qwen2.5:7b fixture=false
08:06:35.307 planner  processing_completed s6102_inbox p0/o4
08:06:35.320 planner  output_published     s6102_tasks p0/o47
08:06:35.322 writer   message_received     s6102_tasks p0/o47
08:06:35.324 writer   processing_started   s6102_tasks p0/o47
08:06:35.325 planner  offset_committed     s6102_inbox committed=5
08:06:39.511 writer   model_result         model=qwen2.5:7b fixture=false
08:06:39.512 writer   processing_completed s6102_tasks p0/o47
08:06:39.517 writer   output_published     s6102_drafts p0/o98
08:06:39.518 reviewer message_received     s6102_drafts p0/o98
08:06:39.521 reviewer processing_started   s6102_drafts p0/o98
08:06:39.522 writer   offset_committed     s6102_tasks committed=48
08:06:43.987 reviewer model_result         model=qwen2.5:7b fixture=false
08:06:43.988 reviewer processing_completed s6102_drafts p0/o98
08:06:43.991 reviewer output_published     s6102_final p0/o12
08:06:43.994 reviewer offset_committed     s6102_drafts committed=99

Final topic: s6102_final | status: approved
Final answer nonempty: True
All stage events preserve the same correlation_id.
Source records: raw/part2/journey.jsonl and processes.json
(base) pragyaapurva@Pragyas-MacBook-Air report-capture-20261102 %
```

Figure 14. The full real-model journey from s6102_inbox to s6102_final.

Each input is committed after its output publication is acknowledged. Full records: raw/part2/journey.jsonl.

Part 3 - Failure results and HW2 comparison

The failure tests use real Kafka/processes and fixture text to isolate delivery behavior. Part 2 uses the real local model.

Failure scenario	Before fix	After fix
Duplicate delivery	2 dictionary rentals / 2 UUIDs; 2 pipeline final outputs.	1 rental, same UUID twice; 1 final logical output.
Writer killed mid-work	Early commit; 0 outputs in the 4-second observation after restart.	Task offset 32 redelivered; 1 draft, then commit 33.
Poison message	Exit at attempt 1; no dead-letter or later output.	3 attempts, 1 dead-letter; later same-partition task succeeds.
Two partitions / Writers	20 unkeyed arrivals across partitions 0 and 1, out of send order.	20 related-key arrivals in partition 1, in order 1-20.

Compared with HW2 function calls

Type	Change with Kafka
Benefit 1	Independent scaling: Writers can share work through partitions in one consumer group.
Benefit 2	Failure isolation: a Writer can restart and recover an uncommitted task without restarting the whole pipeline.
Responsibility 1	Handle replay with stable IDs, duplicate receipts, and commit-after-output behavior.
Responsibility 2	Choose related-entity keys and preserve ordering within each partition.

Matching evidence follows: Figures 15-16 (duplicates), 17 (restart), 18 (poison), and 19-20 (ordering). Raw records are in raw/part3/.

Part 3 - Duplicate delivery

Replaying the same CREATE creates two rentals before idempotency and returns one cached UUID after it.

```
Pragya Apurva | DATA 260 Homework 8
DUPLICATE DELIVERY | idempotency code and actual before/after logs

Worker replay protection: operation + correlation_id + payload fingerprint
src/kafka_service/worker.py
 39         identity = (envelope.operation, envelope.correlation_id)
 40         fingerprint = hashlib.sha256(json.dumps(data, sort_keys=True).encode()).hexdigest()
 41         if self.idempotency and identity in self.responses:
 42             original_fingerprint, response = self.responses[identity]
 43             if original_fingerprint != fingerprint:
 44                 return error_response(payload, "Correlation ID already used with different data")
    ...
 53             return copy.deepcopy(response)
    ...
 70         if self.idempotency:
 71             self.responses[identity] = (fingerprint, copy.deepcopy(response))

BEFORE | idempotency disabled
Replayed correlation_id: ae8a1d34-a200-48f4-849c-8ab47c1f360e
Same CREATE sent twice | request offsets: [53, 54]
Reply 1: success=true rental_id=f3fee838-28c3-4f2f-ac6f-d5bc997eb33a
Reply 2: success=true rental_id=643fea81-20f2-4bdd-a8ed-4ad5bb9c26df
Worker rentals: 2 | distinct UUIDs: 2

AFTER | idempotency enabled
Replayed correlation_id: 79bb5200-aa27-4711-a1a0-5e450689f117
Same CREATE sent twice | request offsets: [55, 56]
Reply 1: success=true rental_id=55c4e748-5a70-466a-a2c1-7fa416dc285f
Reply 2: success=true rental_id=55c4e748-5a70-466a-a2c1-7fa416dc285f
Worker rentals: 1 | distinct UUIDs: 1
08:06:15.571 entity_worker duplicate_cached rental_count=1

Actual pipeline final outputs: before=2 | after=1
Real Kafka/processes; fixture prose is used only for pipeline transport tests.
Evidence: raw/part3/duplicate_before.jsonl and duplicate_after.jsonl
(base) pragyaapurva@Pragyas-MacBook-Air report-capture-20261102 %
```

Figure 15. Idempotency code and the actual before/after dictionary and pipeline replay results.

The worker compares operation, correlation ID, and payload fingerprint. A reused ID with different data returns an error.

Part 3 - Pipeline idempotency

SQLite receipts retain each stage's first result and whether it was published.

```
Pragya Apurva | DATA 260 Homework 8
PIPELINE IDEMPOTENCY | focused source and matching recorded result

src/kafka_agents/reliability.py
 12         self.connection = sqlite3.connect(path, timeout=30)
 13         self.connection.execute("PRAGMA journal_mode=WAL")
 14         self.connection.execute("PRAGMA synchronous=FULL")
 15         self.connection.executescript("""
 16             CREATE TABLE IF NOT EXISTS receipts (
 17                 correlation_id TEXT PRIMARY KEY, fingerprint TEXT NOT NULL,
 18                 output TEXT NOT NULL, published INTEGER NOT NULL DEFAULT 0);
    ...
 34     def remember(self, correlation_id, fingerprint, output):
 35         with self.connection:
 36             self.connection.execute(
 37                 "INSERT OR IGNORE INTO receipts(correlation_id,fingerprint,output) VALUES(?,?,?)",
 38                 (correlation_id, fingerprint, json.dumps(output, sort_keys=True)),
 39             )
 40         # A concurrent Writer cannot replace the first stored logical result.
 41         return self.lookup(correlation_id, fingerprint)

src/kafka_agents/common.py
225         if self.baseline != "no-dedup":
226             try:
227                 cached_data = decode_payload(record.value, self.stage)
228                 cached = self.store.lookup(cached_data["correlation_id"], fingerprint(cached_data))
229                 if cached:
230                     output, published = cached
231                     self.log("duplicate_cached", **source, **ids, already_published=published)
    ...
334             if self.baseline != "no-dedup":
335                 self.store.mark_published(output["correlation_id"])
336         else:
337             self.log("duplicate_suppressed", **source, **ids)
338         if self.baseline != "early-commit":
339             await self.commit(record, consumer, **ids, reason="after_output_ack")

Recorded replay test: same correlation_id sent twice
Final logical outputs: before=2 | after=1
Crash after publish, before commit: final logical outputs=1
Real Kafka/processes; fixture prose. This is application deduplication.
(base) pragyaapurva@Pragyas-MacBook-Air report-capture-20261102 %
```

Figure 16. Receipt code and replay counts: two final logical outputs before, one after.

The publish-before-commit crash test also produced one final logical output. Receipt files must remain intact; Kafka and SQLite are not one atomic transaction.

Part 3 - Writer termination and recovery

Automatic commits are disabled. The restarted Writer receives the same task offset, publishes its output, and then commits the next offset.

```
Pragya Apurva | DATA 260 Homework 8
WRITER DEATH / RESTART | commit code and actual before/after logs

Manual commit follows acknowledged output publication:
src/kafka_agents/common.py
386         enable_auto_commit=False,
    ...
172     async def commit(self, record, consumer, **fields):
173         position = record.offset + 1
174         try:
175             await consumer.commit({TopicPartition(record.topic, record.partition): position})
    ...
312         elif not published:
313             key = str(output.get("rental_id") or output["correlation_id"]).encode()
314             metadata = await producer.send_and_wait(
315                 STAGES[self.stage][1], json.dumps(output).encode(), key=key
316             )
    ...
338         if self.baseline != "early-commit":
339             await self.commit(record, consumer, **ids, reason="after_output_ack")

BEFORE | early commit (failure baseline)
correlation_id: hw8-20261102T080502Z-death-before-120533
Killed PID: 87957 | restarted PID: 87958
08:05:24.839 PID 87957 message_received     s6102_tasks p0/o31
08:05:24.840 PID 87957 offset_committed     committed=32 baseline_before_processing
08:05:24.841 PID 87957 processing_started   s6102_tasks p0/o31
Observed draft outputs: 0 | observation window: 4 seconds

AFTER | commit after output acknowledgement
correlation_id: hw8-20261102T080502Z-death-after-6648ba
Killed PID: 87975 | restarted PID: 87978
08:05:39.101 PID 87975 message_received     s6102_tasks p0/o32
08:05:39.101 PID 87975 processing_started   s6102_tasks p0/o32
08:05:49.092 PID 87978 message_received     s6102_tasks p0/o32
08:05:49.093 PID 87978 processing_started   s6102_tasks p0/o32
08:05:50.094 PID 87978 processing_completed s6102_tasks p0/o32
08:05:50.102 PID 87978 output_published     s6102_drafts p0/o56
08:05:50.110 PID 87978 offset_committed     committed=33 after_output_ack
Observed draft outputs: 1 | observation window: 15 seconds

AFTER: the same task offset is redelivered to the restarted Writer.
Real Kafka/processes; deterministic fixture prose isolates transport behavior.
Evidence: raw/part3/consumer_death_before.jsonl and consumer_death_after.jsonl
(base) pragyaapurva@Pragyas-MacBook-Air report-capture-20261102 %
```

Figure 17. Before/after process IDs, redelivery, output publication, and commit timing.

Task offset 32 reached PIDs 87975 and 87978. Publication at 08:05:50.102 UTC preceded commit 33 at 08:05:50.110.

Part 3 - Poison message and dead-letter handling

plan=null fails validation. After three total attempts, the record moves to s6102_dead_letter and the next valid task succeeds.

```
Pragya Apurva | DATA 260 Homework 8
POISON MESSAGE | bounded retries, dead-letter code and actual logs

src/kafka_agents/common.py
235             attempts = self.store.attempts(source_key)
236             if attempts >= 3:
237                 break
238             attempt = self.store.next_attempt(source_key)
    ...
285                 if attempt < 3:
286                     await asyncio.sleep(self.retry_backoff)
    ...
296                 "original": diagnostic,
    ...
301             metadata = await producer.send_and_wait(DEAD_LETTER, json.dumps(dead).encode())

BEFORE | no retry / no dead-letter handling
Poison correlation_id: hw8-20261102T080502Z-poison-before-33f6ca
Malformed plan=null at task offset 33 | valid later task offset 34
08:05:50.356 processing_failed     attempt=1/3 ValueError: plan must be a nonempty string
Dead-letter records: 0 | later valid draft outputs: 0
Unhandled error exit code: 1

AFTER | three attempts, dead-letter, then continue
Poison correlation_id: hw8-20261102T080502Z-poison-after-4838d1
Malformed plan=null at task offset 35 | valid later task offset 36
08:05:55.549 processing_failed     attempt=1/3 ValueError: plan must be a nonempty string
08:05:55.751 processing_failed     attempt=2/3 ValueError: plan must be a nonempty string
08:05:55.952 processing_failed     attempt=3/3 ValueError: plan must be a nonempty string
08:05:55.954 dead_letter_published s6102_dead_letter p0/o1
08:05:55.955 offset_committed      task o35 -> committed=36
08:05:56.156 output_published      s6102_drafts p0/o57
08:05:56.157 offset_committed      task o36 -> committed=37
Dead-letter records: 1 | later valid draft outputs: 1
DLQ retains original plan: null | source: {'offset': 35, 'partition': 0, 'topic': 's6102_tasks'}
DLQ attempts: 3 | error: ValueError: plan must be a nonempty string

Real Kafka/processes; deterministic fixture prose isolates transport behavior.
Evidence: raw/part3/poison_before.jsonl and poison_after.jsonl
(base) pragyaapurva@Pragyas-MacBook-Air report-capture-20261102 %
```

Figure 18. Retry/dead-letter code and actual attempts, publication, commits, and later output.

The dead-letter record retains the original input, source offset, attempt count, and error. It is acknowledged before the failed input is committed.

Part 3 - Ordering without a rental key

VERIFY_SEED 266102 generates the delay schedule for 20 related messages. The unkeyed baseline explicitly alternates two partitions, with two Writers in one group.

```
Pragya Apurva | DATA 260 Homework 8
UNKEYED BASELINE | 20 related messages, two Writers

VERIFY_SEED: 266102 | same rental: synthetic-rental-6102
Group: hw8-20261102T080502Z-ordering-unkeyed
Distribution: controlled explicit alternating partition; no key
scripts/hw08/failure_experiments.py
386         rng = random.Random(VERIFY_SEED)
    ...
412                     partition=(sequence - 1) % 2 if mode == "unkeyed" else None,
413                     key=payload["rental_id"].encode() if mode == "keyed" else None,

writer-A PID 88000 | settled task partition(s): [0]
writer-B PID 88001 | settled task partition(s): [1]

Observed draft arrival order (UTC); sent sequence was 1 through 20:
Order  Seq  Task p/offset  Writer    Draft offset  Arrival time
    1    2  p1/o30         writer-B           58  08:06:02.455
    2    4  p1/o31         writer-B           59  08:06:02.543
    3    6  p1/o32         writer-B           60  08:06:02.612
    4    8  p1/o33         writer-B           61  08:06:02.650
    5    1  p0/o37         writer-A           62  08:06:02.663
    6   10  p1/o34         writer-B           63  08:06:02.700
    7   12  p1/o35         writer-B           64  08:06:02.783
    8   14  p1/o36         writer-B           65  08:06:02.856
    9   16  p1/o37         writer-B           66  08:06:02.901
   10    3  p0/o38         writer-A           67  08:06:02.905
   11   18  p1/o38         writer-B           68  08:06:02.975
   12   20  p1/o39         writer-B           69  08:06:03.021
   13    5  p0/o39         writer-A           70  08:06:03.180
   14    7  p0/o40         writer-A           71  08:06:03.446
   15    9  p0/o41         writer-A           72  08:06:03.734
   16   11  p0/o42         writer-A           73  08:06:04.013
   17   13  p0/o43         writer-A           74  08:06:04.264
   18   15  p0/o44         writer-A           75  08:06:04.495
   19   17  p0/o45         writer-A           76  08:06:04.764
   20   19  p0/o46         writer-A           77  08:06:05.020

All 20 unique sequences received: True | in send order: False
Real Kafka/processes; seeded delays and fixture prose isolate ordering.
Evidence: raw/part3/ordering_unkeyed.csv and ordering_unkeyed_events.jsonl
(base) pragyaapurva@Pragyas-MacBook-Air report-capture-20261102 %
```

Figure 19. All 20 messages arrive once, but their merged order differs from the send order.

Part 3 - Ordering with the rental key

The same delay schedule is rerun with key=rental_id. Kafka places this rental's messages in one partition.

```
Pragya Apurva | DATA 260 Homework 8
KEYED BY RENTAL_ID | 20 related messages, two Writers

VERIFY_SEED: 266102 | same rental: synthetic-rental-6102
Group: hw8-20261102T080502Z-ordering-keyed
Distribution: rental_id key; normal Kafka partition selection
src/kafka_agents/common.py
313             key = str(output.get("rental_id") or output["correlation_id"]).encode()
314             metadata = await producer.send_and_wait(
315                 STAGES[self.stage][1], json.dumps(output).encode(), key=key
316             )

writer-A PID 88021 | settled task partition(s): [0]
writer-B PID 88022 | settled task partition(s): [1]

Observed draft arrival order (UTC); sent sequence was 1 through 20:
Order  Seq  Task p/offset  Writer    Draft offset  Arrival time
    1    1  p1/o40         writer-B           78  08:06:10.562
    2    2  p1/o41         writer-B           79  08:06:10.624
    3    3  p1/o42         writer-B           80  08:06:10.867
    4    4  p1/o43         writer-B           81  08:06:10.954
    5    5  p1/o44         writer-B           82  08:06:11.230
    6    6  p1/o45         writer-B           83  08:06:11.302
    7    7  p1/o46         writer-B           84  08:06:11.567
    8    8  p1/o47         writer-B           85  08:06:11.601
    9    9  p1/o48         writer-B           86  08:06:11.883
   10   10  p1/o49         writer-B           87  08:06:11.930
   11   11  p1/o50         writer-B           88  08:06:12.209
   12   12  p1/o51         writer-B           89  08:06:12.290
   13   13  p1/o52         writer-B           90  08:06:12.543
   14   14  p1/o53         writer-B           91  08:06:12.614
   15   15  p1/o54         writer-B           92  08:06:12.843
   16   16  p1/o55         writer-B           93  08:06:12.886
   17   17  p1/o56         writer-B           94  08:06:13.152
   18   18  p1/o57         writer-B           95  08:06:13.226
   19   19  p1/o58         writer-B           96  08:06:13.481
   20   20  p1/o59         writer-B           97  08:06:13.525

All 20 unique sequences received: True | in send order: True
Real Kafka/processes; seeded delays and fixture prose isolate ordering.
Evidence: raw/part3/ordering_keyed.csv and ordering_keyed_events.jsonl
(base) pragyaapurva@Pragyas-MacBook-Air report-capture-20261102 %
```

Figure 20. All 20 related messages use partition 1 and arrive in sequence 1-20.

Ordering is preserved for this rental. A single rental key uses one partition even when two Writers are running.

Verification and submission evidence

The full verifier tested the resolved hw8 code commit without skip flags. Its checks validate behavior rather than exact model wording.

```
Pragya Apurva | DATA 260 Homework 8
FINAL ACCEPTANCE CHECKS | recorded verifier and test results

Verifier recorded: 2026-11-02T08:06:44.158372+00:00
Tested commit / current hw8 tag: 8caf32f200a8411db513a784e2f3f7073e8339c5
SID4=6102 | PORT_BASE=8702 | PREFIX=s6102 | DOMAIN_ID=6
SEED=6102 | VERIFY_SEED=266102

Recorded acceptance checks:
PASS  tested_ref
PASS  seven_topic_partitions
PASS  rental_create_get
PASS  rental_replay_same_uuid
PASS  clean_domain_errors
PASS  http_role_logins
PASS  http_missing_invalid_token
PASS  http_wrong_password
PASS  authenticated_fastapi_wrapper
PASS  http_reader_permissions
PASS  http_moderator_permissions
PASS  three_process_pipeline
PASS  writer_death_before
PASS  poison_before
PASS  pipeline_duplicate_before
PASS  writer_death_after
PASS  poison_after
PASS  pipeline_duplicate_after
PASS  ordering_unkeyed
PASS  ordering_keyed
PASS  entity_duplicate_before
PASS  entity_duplicate_after
PASS  publish_commit_boundary
PASS  ordering_seed

Verifier total: 24/24 passed | overall: True
Recorded focused pytest: 68 passed, 34 warnings in 1.68s | 0 skipped
Database preservation: 6 table counts and hashes unchanged
Sources: verification.json, runtime/focused-tests.log, raw/database_preservation.json
(base) pragyaapurva@Pragyas-MacBook-Air report-capture-20261102 %
```

Figure 21. 24/24 checks passed; 68 tests passed, zero skipped; six database tables unchanged.

Required artifact	Saved in reports/hw08/
Console and measurements	RUN_LOG.txt; raw/; METRICS.md
AI-use answers	AI_USE.md and this report
Self-check receipt	verification.json
Report and screenshots	report.pdf; screenshots/

The focused tests reported 34 dependency deprecation warnings and no failures. The single-broker setup demonstrates local recovery and ordering, not high availability.

AI_USE

1. What you used an AI assistant for and what you did yourself.

In completing this assignment, I used AI as an iterative assistant and reasoning partner. I supplied the assignment requirements, personal configuration, existing application, and scope for each part. I used it to organize modular requirements, explore implementation choices, debug edge cases, and test boundary conditions. Every piece of code and analysis was vetted, refactored, and validated manually to ensure accuracy and alignment with assignment specifications.

2. One unsuitable AI-produced output

An early AI-generated recovery test stopped as soon as the Writer produced an output. That left the commit order unverified. Publishing an output and committing the input offset are separate actions, so a crash between them can cause the same message to be delivered again.

3. How the problem was detected

I compared the publication and commit events for the same message. The first capture showed output_published but no offset_committed event. The response looked successful, but I checked whether the logs actually proved that processing happened before the commit. The missing event showed that the test had stopped too early to support that claim.

4. What changed and why it works now

The test now waits for the commit before stopping the Writer. I verified the rerun by following task offset 32 through the restart, then checking that output publication came before commit 33, the next offset to read. The replay test separately confirmed one final logical output. This checks recovery and duplicates, since committing after processing can still allow replay.